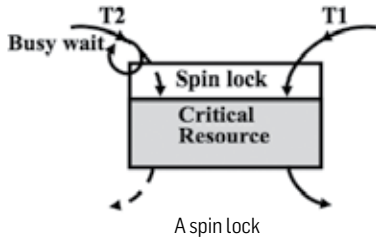


as an abstraction interface between user space processes and the kernel subroutines.

## I/O management

Like memory, the embedded OSes also have to provide an abstraction between the device drivers and the higher level application software. The OS is responsible for providing a universal and a uniform interface for I/O devices that perform a multitude of different operations via system calls. Doing so will allow it to fairly and efficiently manage the I/O among various user space tasks at the same time and the same time protecting the I/O devices as the devices are only accessible via the system call interface. It also has to manage the synchronous and asynchronous communication to and from the I/O devices and the user tasks.



To achieve this, the I/O management scheme typically constitutes of a generic device-driver interface both user processes and device drivers as well as some caching mechanism employed in the middle. The device driver are on the other hand responsible for controlling the actual device on board. Based on the platform, each device driver must implement a specific set of function such as startup, shutdown, enable, disable et al. This varies per platform and usually tightly controlled by commercial embedded OSes. As mentioned earlier the OSes provide an API which control how the higher level processes talk with the device drivers. For the processes higher up in the OS model, these devices act as black boxes and the only method for accessing them is via the API specified by the kernel.

## Timer services

Along with a host of interfaces and functionalities mentioned above the real time OS must also provide the developers will clocks and time services. The time service provided by the operating system is called the system clock. The system clock is maintained by the kernel based on interrupts from a hardware clock. The tricky part here is the time resolution; based on the current technology the hardware clock resolution is somewhere around 1 nanosecond (considering processor clock speed to be 3GHz). But this much fine resolution is not being made available to the user, the user processes get the time with resolution in milliseconds. This discrepancy in time is primarily due to the overhead associated with the hardware clock interrupt ISR, which